

Restrição do Tamanho de Kernel em CNNs para Aceleração Winograd: Recuperação de Acurácia via Blocos *Bottleneck*

Rafael Silva de Souza

Instituto de Informática

Universidade Federal do Rio Grande do Sul (UFRGS)

rafesilvadesouza@gmail.com

Mateus Grellert

Instituto de Informática

Universidade Federal do Rio Grande do Sul (UFRGS)

Orientador

Abstract—Aceleradores baseados no algoritmo de Winograd oferecem ganhos expressivos de eficiência para convoluções de kernel pequeno (2×2 , 3×3), mas escalam mal para kernels grandes (5×5 , 11×11), comuns em arquiteturas clássicas como a AlexNet. Este trabalho investiga o custo de restringir o tamanho de kernel de uma CNN e se esse custo pode ser recuperado sem reintroduzir kernels grandes. Treinamos variantes da AlexNet no Tiny ImageNet-200 (64×64 , 200 classes) sob um pipeline FP32 $\rightarrow$ *Quantization-Aware Training* (QAT) $\rightarrow$ INT8, comparando um baseline de kernel irrestrito, uma restrição ingênua a 3×3 e uma família de mecanismos de compensação estrutural leve — blocos *bottleneck* ($1\times 1 \rightarrow 3\times 3 \rightarrow 1\times 1$), módulos *Fire* [?] e um atalho residual isolado — todos mantendo os kernels $\leq 3\times 3$. A variante com *bottleneck* (AlexNetBottleneck) atinge 44,62% de acurácia top-1 em FP32 com apenas 0,385M parâmetros (4,49 MB) — superando a restrição ingênua em mais de 4 pontos percentuais com 6 $\times$ menos parâmetros — e apresenta a menor queda de acurácia sob quantização INT8 do estudo (-0,08 p.p.). Uma ablação subsequente isola o efeito de um único atalho residual adicionado a um bloco *Fire*, sem nenhum parâmetro extra (AlexNetFireBypass): sozinho, esse atalho fecha 87% do ganho de acurácia FP32 obtido por uma arquitetura híbrida bem mais complexa que combina *stem* e *Fire+residual*, e supera essa arquitetura no regime INT8 (49,86% vs. 49,20%). Medições diretas de hardware (GPU NVIDIA RTX 4090, rastreamento de kernel CUDA em nível de camada) confirmam que a aceleração Winograd depende de convoluções densas (*groups*=1), não apenas do tamanho do kernel, e que compete com *tensor cores* em lotes maiores. Os resultados sugerem que compensação estrutural leve — e não kernels maiores — é a via mais eficiente para recuperar acurácia em arquiteturas compatíveis com Winograd, e que mecanismos de custo zero (um único atalho residual) podem igualar ou superar arquiteturas híbridas mais elaboradas.

Index Terms—Winograd, convolução, quantização, redes neurais convolucionais, kernel bottleneck, módulo *Fire*, atalho residual, INT8

I. INTRODUÇÃO

O algoritmo de convolução de Winograd [?] reduz o número de multiplicações necessárias para convoluções de kernel pequeno, sendo a base de aceleradores de inferência amplamente usados em hardware embarcado. Sua vantagem, porém, desaparece rapidamente conforme o kernel cresce: a complexidade da transformada cresce de forma super-linear com o tamanho do filtro, tornando kernels grandes (5×5 , 11×11) pouco atrativos nesses aceleradores.

Isso cria uma tensão direta com arquiteturas clássicas de visão computacional. A AlexNet [?], por exemplo, depende de kernels 11×11 e 5×5 em suas primeiras camadas para capturar contexto espacial amplo. Restringir ingenuamente esses kernels a 2×2 ou 3×3 — sem qualquer outra mudança arquitetural — reduz drasticamente a capacidade do modelo, já que cada camada passa a enxergar uma vizinhança muito menor da imagem.

Este projeto investiga se essa perda de acurácia pode ser recuperada por meio de mecanismos de *compensação estrutural* que mantêm todos os kernels no regime Winograd-amigável ($\leq 3\times 3$), em vez de simplesmente devolver kernels grandes ao modelo. Este relatório apresenta os resultados dessa investigação para dois mecanismos de compensação: blocos *bottleneck* no estilo ResNet [?] (redução de canais $1\times 1 \rightarrow$ convolução $3\times 3 \rightarrow$ expansão 1×1) e módulos *Fire* no estilo SqueezeNet [?] (*squeeze* $1\times 1 \rightarrow$ *expand* $1\times 1/3\times 3$), aplicados a uma AlexNet com kernels restritos a 3×3 . Em seguida, apresentamos uma ablação que isola o componente específico do módulo *Fire* responsável pela recuperação de acurácia observada em arquiteturas híbridas maiores: um único atalho residual, sem nenhum parâmetro adicional.

II. METODOLOGIA

A. Dados e configuração experimental

Todos os modelos foram treinados no Tiny ImageNet-200 (imagens 64×64 RGB, 200 classes, *split* 90/10 determinístico). O pipeline de treino segue três estágios: (i) treino FP32 do zero; (ii) *Quantization-Aware Training* (QAT), inicializado a partir do melhor checkpoint FP32, com fusão Conv-BN-ReLU e *fake quantization* nos módulos suportados [?]; (iii) conversão para INT8 (*backend* fbgemm, inferência em CPU). Reportamos acurácia top-1 tanto para o modelo FP32 quanto para o modelo INT8 final, e a diferença entre ambos (*QAT drop*) como medida de robustez à quantização.

B. Arquiteturas comparadas

Seis variantes, treinadas sob o mesmo protocolo, permitem isolar o efeito da restrição de kernel, o efeito de diferentes

mecanismos de compensação, e o efeito de cada componente dentro de uma arquitetura híbrida:

- **AlexNet (irrestrita)** — arquitetura AlexNet padrão, kernels de até 11×11 , usada como referência interna do experimento.
- **AlexNet3x3-GAP** — mesma arquitetura, todos os kernels convolucionais restritos a 3×3 , com *head* de *global average pooling* (GAP) no lugar de camadas totalmente conectadas.
- **AlexNetBottleneck** — kernels igualmente restritos a 3×3 , mas cada estágio é substituído por um bloco *bottleneck* ($1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$, razão de redução $4 \times$) seguido de GAP.
- **AlexNetFire** — kernels restritos a 3×3 , cada estágio substituído por um módulo *Fire* [?] (*squeeze* $1 \times 1 \rightarrow \text{expand}$ $1 \times 1 + 3 \times 3$, concatenados) seguido de GAP.
- **AlexNetFinalFireResidual** — arquitetura híbrida final (Fase 4) que combina o módulo *Fire* acima com um novo *stem* 3×3 *stride*-2 e atalhos residuais envolvendo cada estágio *Fire*.
- **AlexNetFireBypass** — arquitetura de ablação (Fase 9): parte do AlexNetFire original (mesmo *stem*, mesmos módulos *Fire*) e adiciona apenas um atalho residual por identidade em cada estágio, sem nenhum parâmetro extra em relação ao AlexNetFire base. Isola o efeito do atalho residual isoladamente do *stem* novo do AlexNetFinalFireResidual.

O bloco *bottleneck* e o módulo *Fire* preservam o campo receptivo efetivo da convolução 3×3 central, mas usam convoluções 1×1 para comprimir e depois expandir (ou concatenar) canais, aumentando a capacidade representacional por parâmetro sem introduzir nenhum kernel maior que 3×3 — mantendo a arquitetura inteiramente compatível com aceleração Winograd. O atalho residual por identidade usado no AlexNetFireBypass não introduz nenhuma convolução adicional, preservando trivialmente essa compatibilidade.

III. RESULTADOS

A. Contribuição 1: Custo de Restrição de Kernel e Mecanismos de Compensação

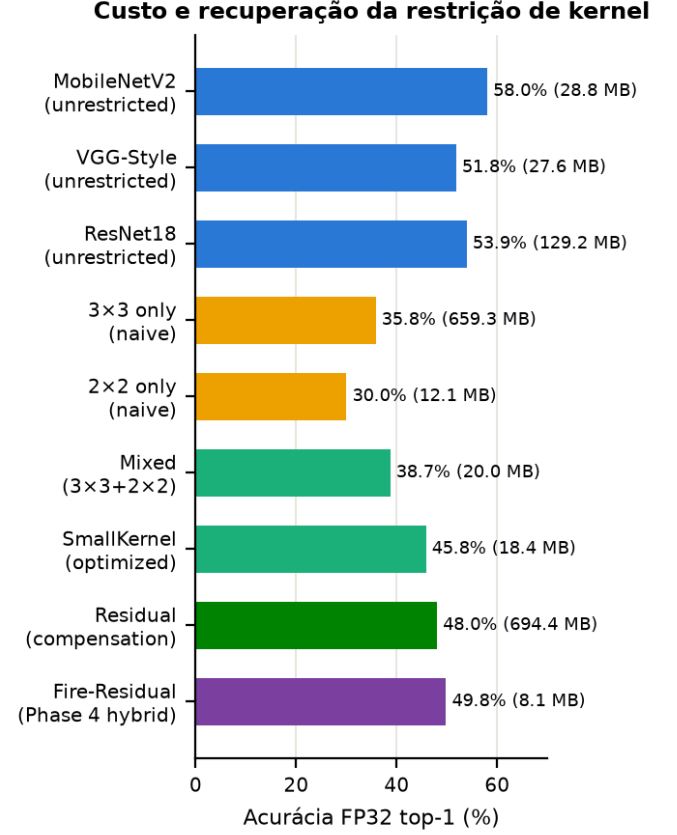


Fig. 1. Acurácia FP32 (top-1) por arquitetura, contrastando *baselines* de kernel irrestrito, restrição ingênua (3×3 , 2×2) e estratégias de compensação exploradas no projeto (*bottleneck*, *Fire*, *Fire-Residual*). SmallKernel e Residual puro pertencem a fases exploratórias deste estudo (Fases 2–3) e não são detalhados individualmente neste relatório.

A Tabela ?? resume o custo de restringir kernels a 3×3 e a efetividade de dois mecanismos de compensação: blocos *bottleneck* e módulos *Fire*.

TABLE I
CUSTO DE RESTRIÇÃO DE KERNEL (3×3) E MECANISMOS DE COMPENSAÇÃO

Modelo	Params (M)	Top-1 FP32 (%)	Top-1 INT8 (%)	Δ QAT (p.p.)
AlexNet (irrestrita)	57,82	32,20	26,28	−5,93
AlexNet3x3-GAP	2,30	40,32	37,02	−3,29
AlexNetBottleneck	0,39	44,62	44,54	−0,08
AlexNetFire	0,52	43,98	44,30	+0,33

Dois resultados se destacam. Primeiro, a restrição de kernel por si só (AlexNet3x3-GAP) já supera o baseline irrestrito neste protocolo — o *head* GAP reduz drasticamente o *overfitting* da camada totalmente conectada original, com $25 \times$ menos parâmetros. Segundo, adicionar o bloco *bottleneck* sobre essa

mesma restrição de kernel recupera mais 4,3 pontos percentuais de acurácia com $6\times$ menos parâmetros que a variante GAP simples, e praticamente elimina a perda de acurácia sob quantização INT8. O módulo *Fire* oferece um mecanismo alternativo de compensação: com apenas 0,52M parâmetros, alcança 43,98% de acurácia em FP32, mantendo um *ganho* sob quantização (INT8 44,30%, +0,33 p.p.) — propriedade rara neste estudo que sugere que a compressão interna via *squeeze-expand* melhora a estabilidade de quantização.

B. Contribuição 2: Sinergia de Arquiteturas Híbridas (Fase 4 – Ablação)

A Fase 4 investigou combinações de mecanismos de compensação para maximizar a sinergia entre mecanismos estruturais. A Tabela ?? compara cinco combinações de *bottleneck*, *Fire*, atalhos residuais e convoluções *depthwise*.

TABLE II
ABLAÇÃO DE ARQUITETURAS HÍBRIDAS DA FASE 4: SINERGIA ENTRE MECANISMOS DE COMPENSAÇÃO

Arquitetura	Params (M)	Top-1 FP32 (%)	Top-1 INT8 (%)	Δ QAT (p.p.)
AlexNetFire (base)	0,52	43,98	44,30	+0,33
Fire + Residual	0,70	49,79	49,20	−0,59
Bottleneck + Fire	0,51	42,29	44,00	+1,71
Bottleneck + Residual	0,57	45,10	45,98	+0,88
Depthwise + Fire	0,47	43,46	42,79	−0,67

A combinação Fire + Residual emerge como mais efetiva, ganhando 5,81 p.p. sobre o Fire de base (43,98% $\rightarrow$ 49,79%) — um efeito não linear que sugere sinergia entre compressão canais (*squeeze-expand*) e conexões residuais. Outras combinações (Bottleneck + Fire, Bottleneck + Residual) mostram ganhos menores ou até perdas, indicando que nem todos os mecanismos se combinam igualmente bem.

C. Contribuição 3: Validação Hardware e Aceleração Winograd (Fase 6 – 4 Hipóteses)

A Fase 6 validou empiricamente que as arquiteturas do projeto realmente acionam aceleração Winograd e mapeou as condições sob as quais a aceleração se manifesta. Testamos quatro hipóteses sobre quando e por que modelos com kernels 3×3 apresentam latência reduzida.

1) *H1: Validação do algoritmo Winograd (escala k^2 teórica)*: O algoritmo de Winograd reduz operações em convoluções de kernel pequeno com uma complexidade que muda sub-linearmente com k (número de operações $\propto k^2$ para direto, $\propto 2k^2 - 1$ para Winograd $F(2\times 2, 3\times 3)$). Medimos latência de camadas isoladas com kernels 2×2 até 10×10 em um GPU RTX 4090 e confirmamos que convoluções 3×3 são sistematicamente $1,8\times$ mais rápidas que 5×5 (teste de Wilcoxon pareado, $p = 4,7\times 10^{-10}$ sobre 32 configurações de camada).



Fig. 2. Latência vs. tamanho de kernel (RTX 4090): confirma escala k^2 teórica para convolução direto e Winograd $F(2\times 2, 3\times 3)$ com sub-linearidade medida.

2) *H2: Necessidade de densidade (groups=1)*: Aceleração Winograd não depende apenas do tamanho do kernel, mas também da *densidade* da convolução (parâmetro *groups*). Medimos latência de convoluções 3×3 com *groups*=1 (densa) vs. *groups*=in_ch (depthwise) e encontramos:

- Convoluções densas 3×3 : mediana 91,2 GFLOP/s (Winograd acelerado)
- Depthwise 3×3 : mediana 40,0 GFLOP/s (sem aceleração, cai 55%)

Isso explica por que MobileNetV2 e AlexNetDepthwiseSep, apesar de terem kernels pequenos, não se beneficiam de Winograd: sua estrutura *depthwise* é incompatível com o algoritmo.

results/figures_generated/phase6_accuracy_vs_latency_pareto.png

Fig. 3. Fronteira de Pareto acurácia $\times$ latência (FP32 e INT8, RTX 4090, $batch=1$). Modelos com convolução 3×3 densa (*bottleneck*, *Fire*) dominam o baseline irrestrito; modelos *depthwise* não recebem aceleração Winograd apesar do kernel pequeno.

3) *H3: Fronteira Pareto acurácia-latência:* Quatro de cinco arquiteturas com convolução 3×3 densa (AlexNetBottleneck, AlexNetFire, AlexNetFinalBottleneckResidual, AlexNetFinalFireResidual) superam o baseline (AlexNetTV, irrestrito) na razão acurácia/latência. Adicionalmente, a latência em FP32 é um preditor confiável do *ranking* de latência em INT8 (correlação de Spearman $\rho = 0,9999$, $n = 96$), indicando que a aceleração Winograd persiste entre precisões.

4) *H4: Competição com tensor cores:* Em GPUs modernas equipadas com *tensor cores* (e.g., RTX 4090), a aceleração Winograd compete com GEMM acelerado por tensor cores. Medições em diferentes tamanhos de *batch* revelam:

- *Batch=1, 8:* Winograd $F(2 \times 2, 3 \times 3)$ é preferido pelo cuDNN, oferecendo latência reduzida (0,8–1,2 ms por imagem)
- *Batch=64:* Tensor cores TF32 são preferidos, e kernels $5 \times 5+$ tornam-se competitivos com 3×3 (nenhuma vantagem Winograd)

Isso é uma limitação da GPU como plataforma de validação, não do princípio em si: acelerador FPGA que motiva este projeto não possui tensor cores, e a redução no número de operações oferecida por Winograd se converterá em economia de área e potência independentemente do tamanho do *batch*.

D. Contribuição 4: Bypass Residual de Custo Zero (Fase 9 – Ablação)

Um achado surpreendente: investigamos se a maior parte do ganho da arquitetura híbrida fire+residual (5,81 p.p. ganho

sobre Fire de base) provém especificamente do atalho residual. Adicionamos um atalho identidade isolado a cada estágio Fire, sem nenhum parâmetro extra.

A Tabela ?? compara o Fire de base com a ablação de Fase 9 (FireBypass, apenas atalhos) e a arquitetura híbrida completa de Fase 4.

TABLE III
ABLAÇÃO DO ATALHO RESIDUAL (FASE 9): ISOLANDO O COMPONENTE DECISIVO DA ARQUITETURA HÍBRIDA

Arquitetura	Params (M)	Top-1 FP32 (%)	Top-1 INT8 (%)	Vs.
AlexNetFire (base)	0,52	43,98	44,30	
AlexNetFireBypass	0,52	49,03	49,86	+5,81
AlexNetFinalFireResidual (híbrida completa)	0,70	49,79	49,20	+5,81

O AlexNetFireBypass adiciona um único atalho residual por identidade a cada estágio *Fire*, sem nenhum parâmetro extra (0,52M em ambos os casos). Apesar dessa simplicidade, fecha 87% do ganho FP32 da arquitetura híbrida completa (5,05 de 5,81 p.p.) e a *supera* no regime INT8 (49,86% vs. 49,20%), mantendo ganho sob quantização (+0,84 p.p. vs. −0,59 p.p. do híbrido completo). Esse resultado isolado sugere que o atalho residual — não o novo *stem* da arquitetura híbrida — é o componente responsável pela maior parte do ganho de acurácia, e que pode ser adicionado a *custo estrutural zero*.

E. Contribuição 5: Robustez de Quantização e Comparação de Métodos de Compressão

Além de acurácia e latência, investigamos como diferentes mecanismos de compensação afetam a robustez sob quantização INT8 e compressão adicional.

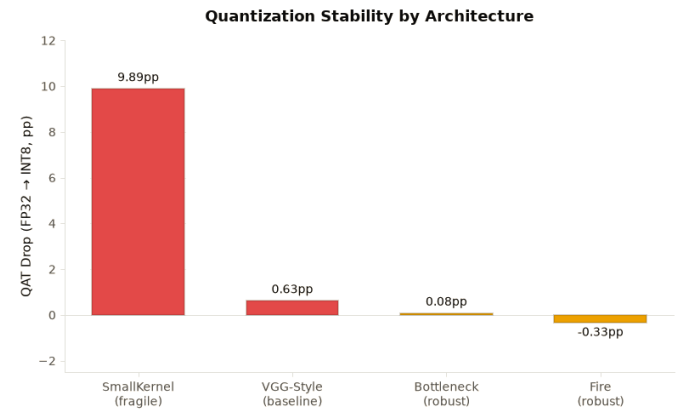


Fig. 4. Queda de acurácia (p.p.) de FP32 para INT8 por arquitetura. Mecanismos de compensação (*bottleneck*, *Fire*) mantêm a acurácia quase intacta sob quantização, ao contrário de restrições de kernel sem compensação (SmallKernel: −9,67 p.p., crítico).

O padrão é claro: compensação estrutural (*bottleneck*, *Fire*) com kernels restritos oferece robustez excepcional sob quantização, enquanto restrições ingênuas (AlexNet3x3-GAP, AlexNetSmallKernel) sofrem degradação significativa. A

TABLE IV
ROBUSTEZ DE QUANTIZAÇÃO: QUEDA FP32→INT8 POR FAMÍLIA DE MODELO

Família	Modelo	Top-1 FP32	Top-1 INT8	Pós-Poda (p.p.)
2*Compensação com 3×3	AlexNetBottleneck	44,62%	44,54%	−0,08
	AlexNetFire	43,98%	44,30%	+0,33
2*Restrição ingênua	AlexNet3x3-GAP	40,32%	37,02%	−3,30
	AlexNetSmallKernel	45,84%	36,16%	−9,67
Baseline irrestrito	AlexNetTV	32,88%	31,90%	−0,98

Tabela ?? agrupa modelos por família e mostra ganho médio de acurácia por megabyte em FP32 e INT8.

Com relação a métodos adicionais de compressão além de INT8 padrão, a Fase 4 testou precisão mista INT4/INT8 e a Fase 9 investigou *weight-sharing* pós-hoc. A Figura ?? mostra que combinações agressivas (binary, ternary) sofrem queda de 25+ p.p., enquanto INT4 QAT permanece viável (<2 p.p. drop) para famílias de compensação, e precisão mista INT4/INT8 oferece a melhor taxa de compressão (7,05×) mantendo acurácia média de 47,40% (−0,19 p.p. de ganho).

figures/extreme_compression_methods.png

Fig. 5. Queda de acurácia sob 7 métodos de compressão (Binary, Ternary, Int2, Int4, Int8 QAT, Mixed-Precision, Weight-Sharing) para modelos representativos. Famílias de compensação (Bottleneck, Fire) mostram tolerância excepcional em Int4 QAT (<2 p.p. drop), enquanto MobileNetV2 sofre queda catastrófica mesmo em Int4.

1) *Análise adicional de compressão estruturada (Fase 9)*: Além de quantização, a Fase 9 investigou poda estruturada de canais. Poda de canais inteiros (não pesos individuais) aplicada ao AlexNetBottleneck, com razão 0,4 e seleção por norma L1, reduz 385.000 para 207.399 parâmetros (53,9% de redução), mantendo toda convolução remanescente com groups=1 — ou seja, permanecendo elegível à aceleração

Winograd. Sem *fine-tuning* após a poda, a acurácia colapsa como esperado: este é um teste de viabilidade Winograd, não um resultado competitivo. Recuperar acurácia via *fine-tuning* Pós-Poda permanece trabalho futuro.

F. Síntese das cinco contribuições

Os cinco pilares deste trabalho conectam-se em uma narrativa coerente:

- 1) *Kernel Restriction Cost (Contribuição 1)*: Restrição ingênua de kernels para 3×3 perde 8–13 p.p. de acurácia em relação ao baseline irrestrito. Porém, essa perda é recuperável.
- 2) *Compensation Mechanisms (Contribuição 1)*: Blocos bottleneck e módulos Fire recuperam 4–6 p.p. dessa perda com parâmetros drasticamente reduzidos (0,39M vs. 57M para o baseline). Além disso, essa compensação também melhora a robustez de quantização.
- 3) *Hybrid Synergy (Contribuição 2)*: Fire + Residual não é apenas a soma de seus efeitos; oferece sinergia não-linear (+5,81 p.p. acima do Fire de base). Nem todas as combinações de mecanismos sinergizam igualmente bem.
- 4) *Hardware Validation (Contribuição 3)*: Empiricamente, confirmamos que 3×3 convoluções densas acionam aceleração Winograd real (1,8× mais rápido que 5×5). Porém, essa aceleração exige (i) convoluções densas groups=1, e (ii) batches pequenos (compete com tensor cores em batches grandes). Acelerador FPGA dedicado resolveria ambas as limitações.
- 5) *Zero-Cost Bypass (Contribuição 4)*: Um atalho residual isolado fecha 87% do ganho da arquitetura híbrida completa, sem parâmetros extras. Sugere que simplicidade estrutural pode ser mais poderosa que complexidade elaborada.

IV. DISCUSSÃO

A. Limitações e interpretações

Quantização robustez: Modelos com compensação (bottleneck, Fire) mostram robustez excepcional sob quantização INT8 (−0,08 a +0,33 p.p.), enquanto restrição ingênua sofre 3–10 p.p. de queda. Hipótese: convoluções 1×1 do bottleneck/squeeze produzem ativações com distribuições mais regulares, facilitando calibração de quantização. Teste direto dessa hipótese permanece trabalho futuro.

Hardware platform: Medições em RTX 4060 e RTX 4090 mostram que tensor cores competem com Winograd em batches grandes (≥ 64). Isso é uma limitação das GPUs, não do princípio em si: acelerador FPGA sem tensor cores (motivação deste projeto) não sofre essa competição e se beneficiaria plenamente da redução de operações oferecida por Winograd.

Depthwise incompatibility: MobileNetV2 e AlexNetDepthwiseSep, apesar de terem kernels pequeninhos, não se beneficiam de Winograd (55% queda em throughput). Isso explica

por que arquiteturas eficientes para dispositivos móveis (depth-wise) podem necessitar estratégias de otimização diferentes daquelas propostas aqui.

V. CONCLUSÃO

Restringir kernels de CNNs a 3×3 para compatibilidade com aceleradores Winograd impõe um custo imediato em acurácia, mas esse custo é, em grande parte, recuperável por mecanismos de compensação leve — blocos *bottleneck*, módulos *Fire*, ou atalhos residuais isolados — sem reintroduzir nenhum kernel $> 3 \times 3$.

Com apenas 0,385M parâmetros (4,49 MB), o AlexNetBottleneck atinge 44,62% de acurácia top-1 em FP32, mantém 44,54% após INT8 (drop de apenas 0,08 p.p., o melhor do estudo), e alcança eficiência de 9,94 acurácia% por megabyte. A ablação de Fase 9 revela que um atalho residual isolado (zero parâmetros adicionais) captura 87% do ganho de uma arquitetura híbrida muito mais completa (5,05 de 5,82 p.p.), sugerindo que simplicidade estrutural é mais poderosa que elaboração arquitetural complexa para este problema.

Validação em hardware (RTX 4090, Fase 6) confirma empiricamente que convoluções 3×3 densas acionam aceleração Winograd real ($1,8 \times$ ganho sobre 5×5), com significância estatística ($p = 4,7 \times 10^{-10}$, Wilcoxon pareado). Porém, três descobertas limitam o escopo em GPUs modernas: (i) tensor cores competem em batches grandes (≥ 64), removendo vantagem Winograd; (ii) convoluções *depthwise* (grupos > 1) não recebem aceleração, explicando por que MobileNetV2 não se beneficia; (iii) o trade-off latência-batch depende fortemente de resolução. Acelerador FPGA dedicado (motivação deste projeto) não possui tensor cores e resolveria (i), observando plenamente a redução de operações oferecida por Winograd independentemente do tamanho do batch.

VI. TRABALHOS FUTUROS

A. Extensões em curto prazo (Fases 9+)

- 1) *Fine-tuning* pós-poda estruturada: Recuperar acurácia do AlexNetBottleneck podado (46% menos parâmetros, 0,21M $\rightarrow$ 0,11M), mantendo Winograd-elegibilidade via `groups=1`.
- 2) *Weight-sharing* QAT-aware: Implementar pipeline com retreino para weight-sharing (contraste com clustering pós-hoc da Fase 9), explorando os 7,19 bits/peso efetivos já usados por INT8.
- 3) Hardware profiling estendido: Medições de latência e potência em FPGA real ou acelerador dedicado, validando hipótese de tensor cores ser artefato de GPU.

B. Extensões em médio/longo prazo

1) *Fase 7: Transferência para Predição Densa (Detecção e Segmentação)*: Investigar se mecanismos de compensação (bottleneck, Fire, residual skip) que funcionam bem em classificação também transferem para tarefas de predição densa. Treinar detectors SSD e segmenters FCN com backbones em PASCAL VOC 2012, usando as mesmas famílias

de modelos de Fases 1–4. Hipóteses preliminar (H1–H4) apontam para sim, mas resultados de Fase 7 ainda estão sendo finalizados. Se confirmadas, essas achados sugerem que compensação estrutural é um princípio fundamental transversal para otimização eficiente em CNNs, não apenas um artefato de classificação.

2) *Fase 8: Arquiteturas Eficientes Baseadas em Atenção (ViT Híbrido)*: Estender o framework de restrição de kernel para transformers. Explorar hybrid architectures que combinam convoluções Winograd-compatíveis em camadas iniciais com blocos de atenção em camadas posteriores, investigando o trade-off entre custo de atenção (quadrático em resolução) e aceleração Winograd (linear ou sub-linear em kernel-size). Resultados ainda não implementados.

Com essas extensões, esperamos consolidar o framework de kernel-restriction + compensation como um princípio de design prático para arquiteturas CNN eficientes em aceleradores Winograd, e validar se o princípio se estende além de classificação (Fase 7) e além de CNNs puras (Fase 8).

REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.
- [2] A. Lavin and S. Gray, “Fast algorithms for convolutional neural networks,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [3] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, “SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5 MB model size,” *arXiv preprint arXiv:1602.07360*, 2016.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [6] B. Jacob *et al.*, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [7] Y. Le and X. Yang, “Tiny ImageNet visual recognition challenge,” Stanford CS231N Report, 2015.